Security Audit Report

Reserve Updates

v1.2

March 12, 2026

Table of Contents

Table of Contents 2
License 4
Disclaimer 5
Introduction 6
Purpose of This Report 6
Codebase Submitted for the Audit 6
Methodology 7
Functionality Overview 7
How to Read This Report 8
Code Quality Criteria 9
Summary of Findings 10
Detailed Findings 12
1. Permit signatures from a previous era can be replayed after an era change to gain allowance over the new stRSR balance 12
2. Force-settling a trade does not reset tradeEnd, temporarily blocking recollateralization 12
3. Cancelling an unstake request with a zero RSR amount leaves phantom drafts in totalDrafts 13
4. Delegation state persists across era changes, carrying over stale delegatees and enabling signature replay 14
5. Stale sell token allowance persists after closing a trusted fill 14
6. Dust-amount unstakes create zero-value draft queue entries 15
7. Aerodrome and Curve gauge wrapper constructors lack validation of the provided tokens 15
8. Lack of zero address check in redeemCustom 16
9. Missing destination validation in _setDistribution 16
10. CFiatV3Wrapper.multiplier has no input validation 17
11. Duplicate error code Err(5) in Upgrade4_2_0 17
12. Missing events for registry state changes 18
13. Deprecation of versions and assets is irreversible 18
14. deprecateVersion does not verify version existence 19
15. RoleRegistry uses deprecated _setupRole 19
16. Registry references could be declared immutable for gas savings 20
17. The trusted filler registry, once initially set, is permanently immutable 20
18. TrustedFillerRegistrySet event has inconsistent emission and formatting 21
19. Public _bidAmount function uses underscore prefix naming convention reserved for internal visibility 21
20. Outdated AerodromeStakingWrapper wrapper name referenced in Aerodrome plugin documentation 22
21. Incorrect NatSpec return documentation in IDeployer 22

22. Misleading comment in GnosisTrade's settle function implies an explicit time check that does not exist 23
23. Miscellaneous documentation concerns 23

License

THIS WORK IS LICENSED UNDER A CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE.

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUE ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

https://oaksecurity.io/
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by ABC Labs to perform a security audit of the Updates to Reserve Protocol from version v3.4.0 to v4.2.0.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	https://github.com/reserve-protocol/protocol
Commit	`8d5b678ce49e38e11b5ae5c54639561fa619e6fb`
Scope	All changes between commits `d96e0cbf6ed57629ee64c2dbc6ae7159debaf509` (v3.4.0) and `8d5b678ce49e38e11b5ae5c54639561fa619e6fb` (v4.2.0), focusing on the P1 implementation contracts. The P0 contracts, mocks, and facades are out of scope.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
    a. Race condition analysis
    b. Under-/overflow issues
    c. Key management vulnerabilities
4. Report preparation

Functionality Overview

Reserve Protocol enables the creation of overcollateralized stablecoins backed by configurable asset baskets, with built-in mechanisms for rebalancing through auctions and revenue distribution. The v4.2.0 upgrade focuses on security and governance hardening through a global reentrancy lock across all components and three new registries that whitelist valid versions, assets, and DAO fee configurations. Key new features include trusted filler support for async auction settlement via CowSwap and an issuance premium mechanism that increases minting costs when backing collateral depegs.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	Medium	-
Code readability and clarity	Medium-High	Consistent unit comments and annotations; some stale NatSpec.
Level of documentation	Medium-High	Comprehensive docs folder; newer features are less documented.
Test coverage	Medium-High	Test coverage is approximately 86% for the core protocol contracts (p1, libraries, mixins), with plugins, registry, and spells having lower coverage as they rely on integration/fork tests.

Summary of Findings

No	Description	Severity	Status
1	Permit signatures from a previous era can be replayed after an era change to gain allowance over the new stRSR balance	Minor	Acknowledged
2	Force-settling a trade does not reset `tradeEnd`, temporarily blocking recollateralization	Minor	Acknowledged
3	Cancelling an unstake request with a zero RSR amount leaves phantom drafts in `totalDrafts`	Minor	Acknowledged
4	Delegation state persists across era changes, carrying over stale delegatees and enabling signature replay	Minor	Acknowledged
5	Stale sell token allowance persists after closing a trusted fill	Informational	Acknowledged
6	Dust-amount unstakes create zero-value draft queue entries	Informational	Acknowledged
7	Aerodrome and Curve gauge wrapper constructors lack validation of the provided tokens	Informational	Acknowledged
8	Lack of zero address check in `redeemCustom`	Informational	Acknowledged
9	Missing destination validation in `_setDistribution`	Informational	Acknowledged
10	`CFiatV3Wrapper.multiplier` has no input validation	Informational	Acknowledged
11	Duplicate error code `Err(5)` in `Upgrade4_2_0`	Informational	Acknowledged
12	Missing events for registry state changes	Informational	Acknowledged
13	Deprecation of versions and assets is irreversible	Informational	Acknowledged
14	`deprecateVersion` does not verify version existence	Informational	Acknowledged
15	`RoleRegistry` uses deprecated `_setupRole`	Informational	Acknowledged
16	Registry references could be declared `immutable` for gas savings	Informational	Acknowledged
17	The trusted filler registry, once initially set, is	Informational	Acknowledged

	permanently immutable		
18	`TrustedFillerRegistrySet` event has inconsistent emission and formatting	**Informational**	**Acknowledged**
19	Public `_bidAmount` function uses underscore prefix naming convention reserved for internal visibility	**Informational**	**Acknowledged**
20	Outdated `AerodromeStakingWrapper` wrapper name referenced in Aerodrome plugin documentation	**Informational**	**Acknowledged**
21	Incorrect NatSpec return documentation in `IDeployer`	**Informational**	**Acknowledged**
22	Misleading comment in `GnosisTrade`'s `settle` function implies an explicit time check that does not exist	**Informational**	**Acknowledged**
23	Miscellaneous documentation concerns	**Informational**	**Acknowledged**

Detailed Findings

1. Permit signatures from a previous era can be replayed after an era change to gain allowance over the new stRSR balance

Severity: Minor

In `contracts/p1/StRSR.sol:126`, the `_nonces` mapping used by the ERC-20 `permit` function is not scoped by `era`. When `beginEra` is called in line `695`, all stake balances and allowances are effectively wiped by incrementing the `era` counter, since both `stakes` and `_allowances` are keyed by `era`. However, `_nonces` remains a simple mapping that persists across era transitions.

This creates a permit replay vector: if a staker signs a `permit` message (with a far-future deadline) in era N and the signature is never submitted on-chain, the nonce remains unconsumed. After an era change, e.g., triggered by a complete RSR seizure in `seizeRSR` or a governance-initiated `resetStakes`, the victim may re-stake in era N+1.

At that point, someone with the old, unused permit signature can submit it. Since the nonce has not incremented, the signature passes validation, and `_approve` in line `916` writes the allowance to `_allowances[era][owner][spender]` for the new era, granting approval for the staker's new balance in era N+1.

The `_PERMIT_TYPEHASH` does not include the era, so the signed data is identical across eras for the same nonce, owner, spender, value, and deadline. This means the signature remains cryptographically valid even though `era` has changed.

Recommendation

We recommend including the current `era` in the permit struct hash so that signatures are bound to a specific era and become invalid after an era transition.

Status: Acknowledged

2. Force-settling a trade does not reset `tradeEnd`, temporarily blocking recollateralization

Severity: Minor

In `contracts/p1/BackingManager.sol:321-324`, the `forceSettleTrade` function allows governance to forcibly settle a trade. The function clears the `tokensOut` mapping for the sold token, removes the trade from `trades`, and decrements `tradesOpen`.

However, the `forceSettleTrade` function does not reset the `tradeEnd` mapping for the settled trade's `TradeKind`. In contrast, the regular `settleTrade` function in line `90` explicitly updates `tradeEnd[kind]` to `block.timestamp` if it is still set to a future value.

Since `forceSettleTrade` omits this step, the stale `tradeEnd` value from the original trade persists.

The `rebalance` function in line `117` requires `tradeEnd[kind] < block.timestamp` before allowing a new trade. If `tradeEnd[kind]` still holds a future timestamp from the force-settled trade, calls to `rebalance` will revert with `"already rebalancing"` until that timestamp passes. This stalls the recollateralization process for up to the full duration of the original trade.

We classify this issue as minor since the `forceSettleTrade` function is governance-only.

Recommendation

We recommend resetting `tradeEnd[kind]` to `block.timestamp` in the `forceSettleTrade` function.

Status: Acknowledged

3. Cancelling an unstake request with a zero RSR amount leaves phantom drafts in `totalDrafts`

Severity: Minor

In `contracts/p1/StRSR.sol:356-400`, the `cancelUnstake` function advances the user's draft pointer via `firstRemainingDraft` in line `380`, then computes the corresponding RSR amount. If `rsrAmount` rounds to zero due to ceiling division, the function returns early in line `388`.

However, this early return occurs before `totalDrafts` is decremented. The user's drafts are consumed but still counted in `totalDrafts`, creating phantom drafts that belong to no account yet inflate the global draft counter and violate the stated invariant that `totalDrafts == sum(draftSum(draft[acct]) for acct in draft)`.

Over time, this can accumulate and cause a minor accounting drift, slightly reducing future paid-out RSR when users withdraw or cancel unstaking. The practical economic impact is still negligible, but the invariant drift is persistent until a draft-era reset.

The `withdraw` function avoids this by updating `totalDrafts` in line `338` before the zero-amount early return in line `339`.

Recommendation

We recommend moving the `totalDrafts` update before the early return in `cancelUnstake`, matching the pattern used in `withdraw`.

Status: Acknowledged

4. Delegation state persists across era changes, carrying over stale delegatees and enabling signature replay

Severity: Minor

In `contracts/p1/StRSRVotes.sol:34`, the `_delegates` mapping is not scoped by `era`. When the `beginEra` function is called, all stake balances are wiped by incrementing the `era` counter, and voting power checkpoints in `_checkpoints[era]` start fresh. However, `_delegates` retains the delegation relationships from the previous era.

When a user re-stakes in the new era, the `_afterTokenTransfer` hook function calls `_moveVotingPower` using `delegates(account)`, which returns the stale delegatee from the prior era. The user's voting power is assigned to this old delegatee even though the user never delegated in the new era.

In `contracts/p1/StRSR.sol:128`, the `_delegationNonces` mapping is also not scoped by `era` and is not reset during `beginEra`. This allows `delegateBySig` signatures from a prior `era` to be replayed after the victim re-stakes. Since the `_DELEGATE_TYPEHASH` does not include the `era` and the nonce remains unconsumed, anyone holding an old delegation signature can redirect the victim's voting power in the new era.

Recommendation

We recommend scoping the `_delegates` and `_delegationNonces` mappings by `era` so that delegation relationships and nonces do not carry over. Additionally, the `era` should be included in the `_DELEGATE_TYPEHASH` to invalidate prior-era signatures.

Status: Acknowledged

5. Stale sell token allowance persists after closing a trusted fill

Severity: Informational

In `contracts/plugins/trading/DutchTrade.sol:326`, the `createTrustedFill` function grants the deployed filler contract a sell-token approval for the full `sellAmt`.

When the trusted fill is subsequently closed through the `_closeTrustedFill` function, the function calls `closeFiller` on the active filler and deletes the `activeTrustedFill` reference. However, it does not revoke the previously granted sell-token allowance. As a result, the filler contract retains a token spending approval for the full sell amount even after it has been closed.

While the practical risk is limited since the filler is a trusted contract deployed through the `TrustedFillerRegistry`, leaving stale allowances is inconsistent with the principle of least privilege and deviates from the expected cleanup behavior.

Recommendation

We recommend revoking the sell-token allowance in the `_closeTrustedFill` function.

Status: Acknowledged

6. Dust-amount unstakes create zero-value draft queue entries

Severity: Informational

In `contracts/p1/StRSR.sol:278-283`, the `unstake` function computes the RSR amount to move from the stake pool to the draft pool. The calculation in line `278` uses ceiling division to determine `newStakeRSR`, and the difference `stakeRSR - newStakeRSR` yields the `rsrAmount` passed to the `pushDraft` function. When `stakeAmount` is very small relative to the current `stakeRate`, this difference can be rounded to zero.

Despite `rsrAmount` being zero, the `pushDraft` function is still called in line `283`, which appends a new `CumulativeDraft` entry to the user's draft queue. These zero-value entries unnecessarily increase the draft queue size.

Recommendation

We recommend adding a check after computing the `rsrAmount` to skip the draft creation when `rsrAmount` is zero.

Status: Acknowledged

7. Aerodrome and Curve gauge wrapper constructors lack validation of the provided tokens

Severity: Informational

In `contracts/plugins/assets/aerodrome/AerodromeGaugeWrapper.sol:22-31`, the `constructor` validates that the reward token matches the `gauge`'s `rewardToken`, but does not verify that the provided `_lpToken` matches the `stakingToken`.

Similarly, in `contracts/plugins/assets/curve/crv/CurveGaugeWrapper.sol:38-46`, the `constructor` performs no validation at all. Neither the reward token nor the LP token is checked against the `gauge`.

A mismatch between the LP token and the gauge's expected staking token would cause the `deposit` function to revert.

Recommendation

We recommend adding checks to ensure both the reward token and the LP token match the expected tokens for the gauge.

Status: Acknowledged

8. Lack of zero address check in `redeemCustom`

Severity: Informational

In `contracts/p1/RToken.sol:257-348`, the `redeemCustom` function does not check that `recipient != address(0)`. In contrast, `redeemTo` in line `195` includes an explicit check: `require(recipient != address(0), "cannot redeem to zero address")`.

If a user accidentally passes `address(0)` as `recipient` in `redeemCustom`, the collateral tokens would be sent to the zero address via `safeTransfer`, effectively burning them permanently. The user's RTokens would be burned, and the backing collateral would be lost. Most ERC20 implementations do not reject transfers to `address(0)`, so the transaction would succeed.

Recommendation

We recommend adding `require(recipient != address(0), "cannot redeem to zero address")` to `redeemCustom` for consistency with `redeemTo`.

Status: Acknowledged

9. Missing destination validation in `_setDistribution`

Severity: Informational

In `contracts/p1/Distributor.sol:240-265`, the `_setDistribution` function validates that `dest` is not `address(0)`, `furnace`, `stRSR`, `rsr`, `rToken`, or `daoFeeRegistry`.

However, it does not prevent setting `dest` to `rsrTrader`, `rTokenTrader`, `backingManager`, `main`, or `distributor` itself. Setting a destination for a revenue trader would create a circular flow, with revenue distributed back to the trader.

Recommendation

We recommend adding validation to prevent setting destinations to any system component address, for example, by checking `require(!main.isComponent(dest), "destination cannot be a component")`.

Status: Acknowledged

10. `CFiatV3Wrapper.multiplier` has no input validation

Severity: Informational

In `contracts/plugins/assets/compoundv3/CFiatV3Wrapper.sol:39, 52`, the `rewardMultiplier` constructor parameter is stored directly as `multiplier` in line `52` with no validation. It is used in reward calculations in lines `208` and `277`.

If `multiplier` were set to an extremely large value, the intermediate multiplication `baseTrackingAccrued * rescaleFactor * multiplier` could overflow `uint256`, causing rewards to be permanently unclaimable. Conversely, if set to 0, rewards would always be zero.

In practice, this value mirrors Compound's `RewardConfig.multiplier`, which is typically `1e18`, so the risk is limited to deployment misconfiguration. Since this is an immutable value, a mistake cannot be corrected without redeploying the wrapper.

Recommendation

We recommend adding bounds validation in the constructor, e.g., `require(rewardMultiplier > 0 && rewardMultiplier <= 1e36, "invalid multiplier")`.

Status: Acknowledged

11. Duplicate error code `Err(5)` in `Upgrade4_2_0`

Severity: Informational

In `contracts/spells/4_2_0.sol:247` and `contracts/spells/4_2_0.sol:253`, the error code `Err(5)` is used for two distinct failure conditions: the `supported[rToken]` check in line `247` (verifying the RToken is in the supported set) and the `MAIN_OWNER_ROLE` validation in line `253` (verifying both the caller and the spell hold `OWNER` on Main). If the spell reverts with `Err(5)`, it is ambiguous which condition failed, making debugging and incident response more difficult.

At the same time, all other error codes in the contract are unique to their respective checks.

Recommendation

We recommend assigning a distinct error code to one of the two conditions to eliminate ambiguity.

Status: Acknowledged

12. Missing events for registry state changes

Severity: Informational

Several registry-related state changes do not emit events:

- In `contracts/p1/Main.sol:71-94`, the setters `setVersionRegistry` (line `71`), `setAssetPluginRegistry` (line `80`), and `setDAOFeeRegistry` (line `89`) modify critical registry references without emitting events.
- In `contracts/registry/AssetPluginRegistry.sol:112`, the `deprecateAsset` function marks assets as deprecated without emitting an event.

This inconsistency with other protocol setters, which emit events such as `WarmupPeriodSet` and `BatchAuctionLengthSet`, prevents off-chain monitoring and indexing services from detecting these state changes without polling.

Recommendation

We recommend adding and emitting events for the registry setters in `Main.sol` and an `AssetDeprecated(address asset)` event in `deprecateAsset`.

Status: Acknowledged

13. Deprecation of versions and assets is irreversible

Severity: Informational

In `VersionRegistry.sol:59-70` and `AssetPluginRegistry.sol:107-113`, the `deprecateVersion` and `deprecateAsset` functions set deprecation flags with no corresponding undo mechanism. If deprecated by mistake, recovery would require redeploying the registry and upgrading `Main`.

If a version or asset is deprecated by mistake (by the owner or the emergency council), there is no way to reverse the action. A new `VersionRegistry` or `AssetPluginRegistry` contract would need to be deployed and all state re-registered. Since `Main.sol` only allows setting each registry reference once (via `setVersionRegistry`, `setAssetPluginRegistry`), this would additionally require upgrading `Main` itself.

Recommendation

We recommend adding an `undeprecateVersion` function to `VersionRegistry` (owner-only) and an `undeprecateAsset` function to `AssetPluginRegistry` (owner-only).

Status: Acknowledged

14. `deprecateVersion` does not verify version existence

Severity: Informational

In `contracts/registry/VersionRegistry.sol:59-70`, the `deprecateVersion` function does not check whether the provided `versionHash` corresponds to a previously registered version (i.e., `deployments[versionHash] != address(0)`). This allows the owner or emergency council to "deprecate" a version hash that was never registered, setting `isDeprecated[versionHash] = true` and emitting a `VersionDeprecated` event for a non-existent version.

Recommendation

We recommend adding a check `require(address(deployments[versionHash]) != address(0))` at the beginning of `deprecateVersion`.

Status: Acknowledged

15. `RoleRegistry` uses deprecated `_setupRole`

Severity: Informational

In `contracts/registry/RoleRegistry.sol:15`, the constructor uses `_setupRole(DEFAULT_ADMIN_ROLE, msg.sender)`. This function was deprecated in OpenZeppelin 4.x in favor of `_grantRole`. While `_setupRole` still works in the currently used OpenZeppelin version (4.9.6), it may be removed in future versions. Other parts of the codebase (e.g., `Auth.sol`) already use `_grantRole`.

Recommendation

We recommend replacing `_setupRole(DEFAULT_ADMIN_ROLE, msg.sender)` with `_grantRole(DEFAULT_ADMIN_ROLE, msg.sender)` for consistency and forward compatibility.

Status: Acknowledged

16. Registry references could be declared `immutable` for gas savings

Severity: Informational

In `contracts/registry/VersionRegistry.sol:16`, `roleRegistry` is declared as a regular state variable, but is only set in the constructor and has no setter. The same applies to `contracts/registry/DAOFeeRegistry.sol:10` (`roleRegistry`) and `contracts/registry/AssetPluginRegistry.sol:13-14` (`versionRegistry` and `roleRegistry`).

Declaring these as `immutable` would save gas on every read by avoiding SLOAD operations.

Recommendation

We recommend declaring the abovementioned constructor-set-only references as `immutable`.

Status: Acknowledged

17. The trusted filler registry, once initially set, is permanently immutable

Severity: Informational

In `contracts/p1/Broker.sol:308-322`, the `_setTrustedFillerRegistry` function prevents changing the registry once set via `require(address(trustedFillerRegistry) == address(0), "trusted filler registry already set")`. However, there is no validation that the `_newFillerRegistry` parameter is not `address(0)` when initially setting it, and no validation that it implements the expected interface.

If governance sets the registry to an incorrect or malicious contract address on the first call, it can never be changed. The only recourse is to keep `trustedFillerEnabled = false` to neutralize the bad registry, but the incorrect address is permanently stored.

Additionally, if `setTrustedFillerRegistry(address(0), true)` is called, the condition `address(trustedFillerRegistry) != _newFillerRegistry` evaluates to false (both are zero), so the function skips the `if` block and only sets `trustedFillerEnabled = true`, resulting in an inconsistent state where the feature is "enabled" but the registry is `address(0)`.

Recommendation

We recommend adding `require(_newFillerRegistry != address(0))` when the registry is being set for the first time, and optionally performing an interface check to validate that the contract responds correctly.

Status: Acknowledged

18. `TrustedFillerRegistrySet` event has inconsistent emission and formatting

Severity: Informational

In `contracts/p1/Broker.sol:321`, the `TrustedFillerRegistrySet` event is emitted on every call to `_setTrustedFillerRegistry`, even when neither the registry address nor the enabled flag changes. The event emission occurs outside both conditional `if` blocks, so a call to `governance.setTrustedFillerRegistry(address(registry), currentEnabled)` would emit a redundant event.

Recommendation

We recommend emitting the event only when a state change occurs.

Status: Acknowledged

19. Public `_bidAmount` function uses underscore prefix naming convention reserved for internal visibility

Severity: Informational

In `contracts/plugins/trading/DutchTrade.sol:472`, the `_bidAmount` function is declared with `public` visibility but uses an underscore prefix in its name. By convention, the underscore prefix denotes `internal` or `private` visibility.

This inconsistency between the naming convention and the actual visibility can lead to confusion, as they may overlook the function when searching for public entry points or mistakenly assume it is not callable externally.

Recommendation

We recommend either renaming the function to `bidAmount` to align with its `public` visibility or changing the visibility to `internal` if external access is not required.

Status: Acknowledged

20. Outdated `AerodromeStakingWrapper` wrapper name referenced in Aerodrome plugin documentation

Severity: Informational

The NatSpec comments in the Aerodrome collateral plugin files reference `AerodromeStakingWrapper` as the expected ERC-20 wrapper, but the actual contract implementation is named `AerodromeGaugeWrapper` in `contracts/plugins/assets/aerodrome/AerodromeGaugeWrapper.sol:9`.

This naming inconsistency appears in the following locations:

- `contracts/plugins/assets/aerodrome/AerodromeVolatileCollateral.sol:23,35`
- `contracts/plugins/assets/aerodrome/AerodromeStableCollateral.sol:11,23`

Recommendation

We recommend updating all references of `AerodromeStakingWrapper` to `AerodromeGaugeWrapper` in the affected NatSpec comments to accurately reflect the implementation.

Status: Acknowledged

21. Incorrect NatSpec return documentation in `IDeployer`

Severity: Informational

In `contracts/interfaces/IDeployer.sol:114`, the `@return` NatSpec comment states "The address of the newly deployed Main instance." The actual implementation in `contracts/p1/Deployer.sol:281` returns `address(components.rToken)`, which is the `RToken` address. The implementation's own NatSpec at `contracts/p1/Deployer.sol:94` correctly states `@return The address of the newly deployed RToken.`

Integrators reading the interface documentation would expect the `Main` address, but would receive the RToken address.

Recommendation

We recommend updating the `@return` NatSpec in `IDeployer.sol:114` to `@return The address of the newly deployed RToken.`

Status: Acknowledged

22. Misleading comment in `GnosisTrade`'s `settle` function implies an explicit time check that does not exist

Severity: Informational

In `contracts/plugins/trading/GnosisTrade.sol:175`, the NatSpec comment for the `settle` function lists `now >= endTime` as one of its checked conditions. However, the function does not perform an explicit timestamp validation. Instead, the time enforcement is implicitly delegated to the externally called `gnosis.settleAuction` function in line `197`, which reverts if the auction has not yet ended.

This discrepancy between the documented checks and the actual implementation can lead to believing that a local time guard exists within the `settle` function. If the behavior of the external `gnosis.settleAuction` call were to change, the absence of an explicit local check could go unnoticed.

Recommendation

We recommend updating the comment to clarify that the `now >= endTime` precondition is implicitly enforced by the external `gnosis.settleAuction` call rather than by a local check within the function.

Alternatively, if an explicit time check is desired for clarity and defense-in-depth, consider adding a local timestamp validation in the settle function.

Status: Acknowledged

23. Miscellaneous documentation concerns

Severity: Informational

The codebase has the following documentation concerns:

- In `docs/recollateralization.md:89`, the documentation states the haircut reduces `RToken.basketsNeeded()` to `basketRange.bottom`. However, `contracts/p1/BackingManager.sol:170` passes `basketsHeld.bottom` (the actual current BU holdings) to `compromiseBasketsNeeded`, not `basketRange.bottom` (a pessimistic post-trading projection). We recommend updating `docs/recollateralization.md` to replace `basketRange.bottom` with `basketsHeld.bottom`.
- In `docs/mev.md:99`, the documentation advises avoiding "the first 40% of the auction." The violation detection in `DutchTrade.sol:238` triggers when `price > bestPrice * 1.5`, which corresponds to the first 20% (the geometric phase). The code's own comment in line 86 confirms "first 20%." We recommend updating `docs/mev.md` to reference "20%" instead of "40%."

- `docs/system-design.md` does not mention any major v4.2.0 additions: DAO fee integration (`DAOFeeRegistry`), issuance premium, trusted filler registry, reweightable baskets with `lastCollateralized` tracking, or asset plugin registry validation. The core design document remains at a pre-v4.2.0 state.
- `docs/reweightable-rtokens.md` does not document: `lastCollateralized` (tracks last fully-collateralized basket nonce), `quoteCustomRedemption` (multi-basket redemption), `forceSetPrimeBasket` (governance basket change without target amount checks), or `requireConstantConfigTargets` (validation skipped by `forceSetPrimeBasket`).

Recommendation

We recommend addressing the abovementioned documentation inconsistencies.

Status: Acknowledged